Dynamic Programming
(continued)

이원형 교수
Dr. Lee, WonHyong
whlee@handong.edu

Matrix-Chain Multiplication (continued)

EXAMPLE

- Let A_1, A_2, A_3, and A_4 have dimensions $20\text{x}10, 10\text{x}50, 50\text{x}5$, and $5\text{x}30$, respectively.
- To get an optimal choice,
  how many cases do we have to try and compare?
  - ✓ For four-matrix case, it is five.
  - ✓ What about n-matrix case?

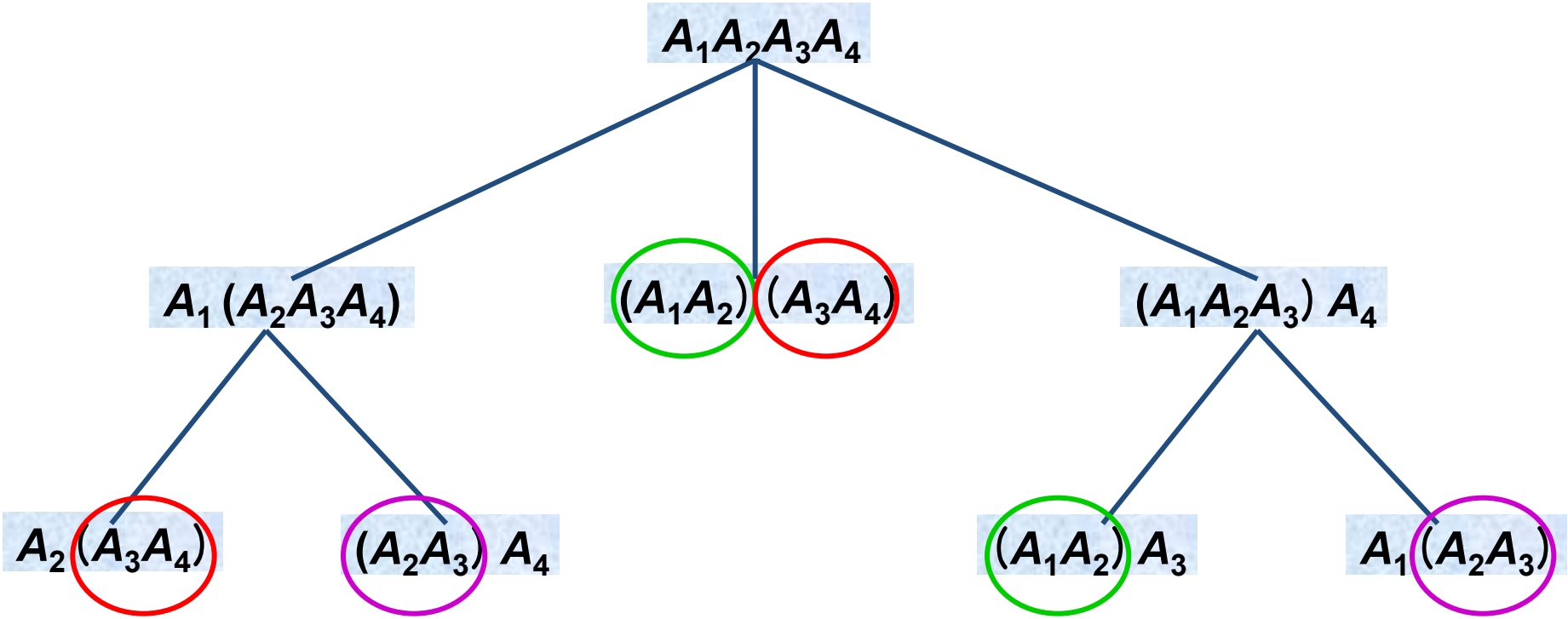
Parenthesizations	Number of multiplications
$(A_1(A_2(A_3A_4)))$	28500
$(A_1((A_2A_3)A_4))$	10000
$((A_1A_2)(A_3A_4))$	47500
$((A_1(A_2A_3))A_4)$	6500
$(((A_1A_2)A_3)\ A_4)$	18000

- **Problem:**
  - ✓ Parenthesize a chained matrix product so as to minimize the number of multiplications

A *brute-force* algorithm that examines **all possible parenthesizations** is computationally infeasible, since the time complexity is

$\Omega(2^n)$

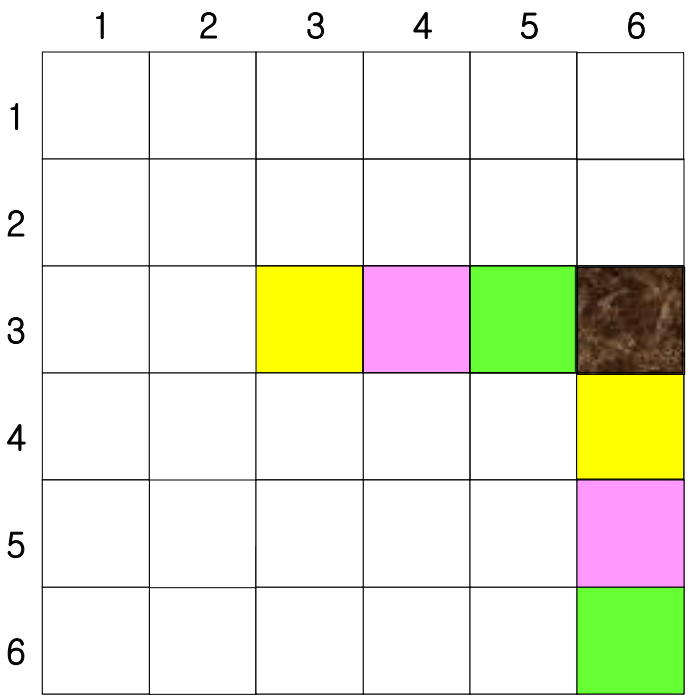
Recalculation: The Source of All Evil



When Can We Apply Dynamic Programming?

- **To apply Dynamic Programming,**
  - ✓ the optimization problem must have two properties:
- **Optimal Substructure**
  - ✓ An optimal solution to an instance (a problem) contains within it optimal solutions to smaller instances of the same problem (subproblems).
- **Optimal Overlapping Subproblems**
  - ✓ A recursive solution to the problem solves certain smaller instances of the same problem <u>over and over again</u>, rather than new subproblems.

MATRIX-CHAIN MULTIPLICATION



$A_3A_4A_5A_6$: [3,6]

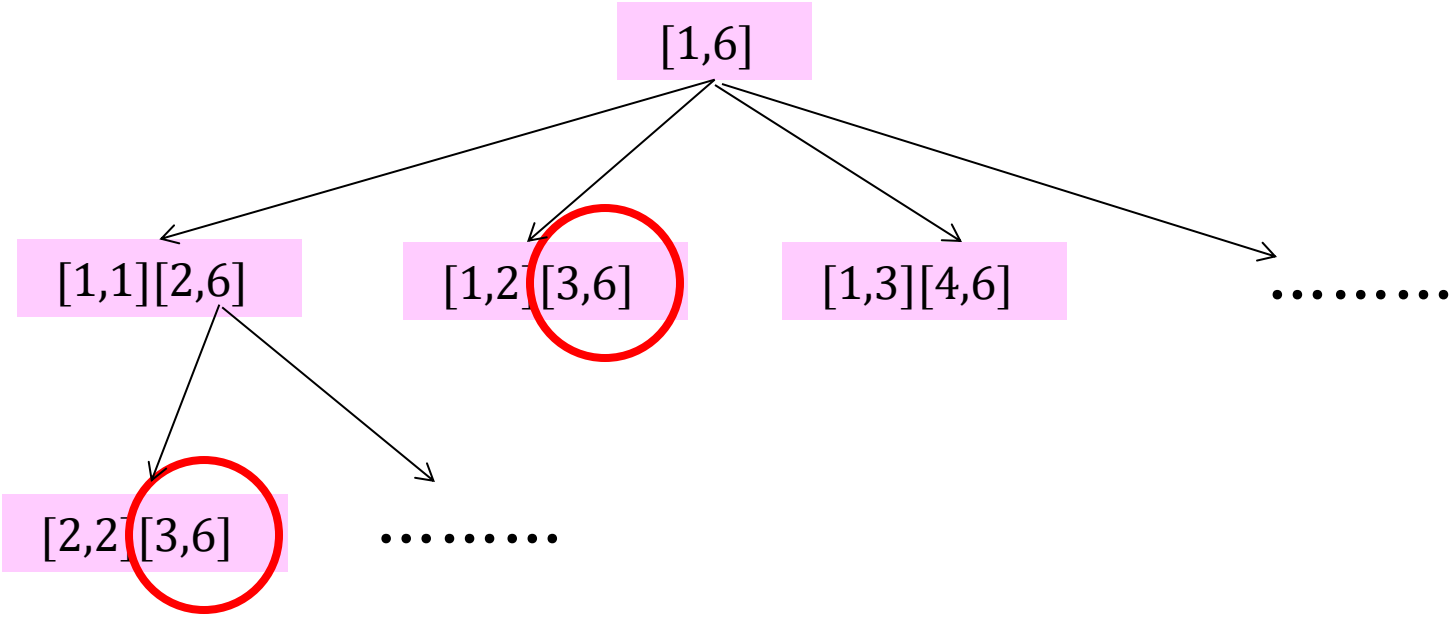
$[3,6] = [3,5] \times [6,6]$

$[3,6] = [3,4] \times [5,6]$

$[3,6] = [3,3] \times [4,6]$

If "$[3,6] = [3,4] \times [5,6]$" is optimal solution,
then we don't have to consider
"$[3,6] = [3,5] \times [6,6]$" and "$[3,6] = [3,3] \times [4,6]$" any more.

Optimal Overlapping Subproblems



Development of DP algorithms

1. Characterize the structure of an optimal solution.
2. Recursively define the value of an optimal solution.
3. Compute the value of an optimal solution in a bottom-up fashion.
4. Construct an optimal solution from computed information.

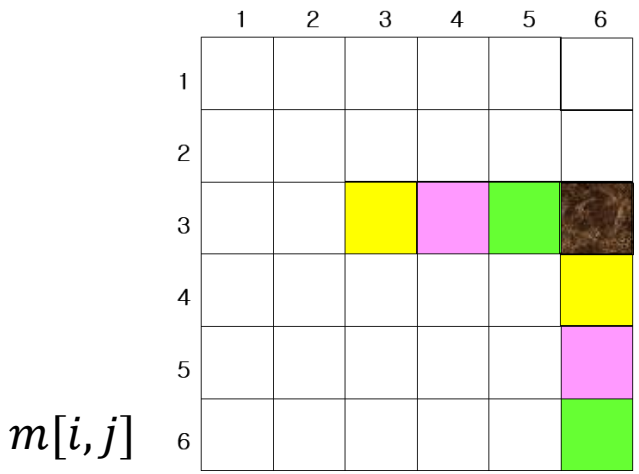
STEP 1

- **Find the optimal substructure**
  - ✓ We can build an optimal solution to an instance of the matrix-chain multiplication problem by
    - splitting the problem into two subproblems (optimally parenthesizing $A_i A_{i+1} \dots A_k$ and $A_{k+1} A_{k+2} \dots A_j$),
    - finding optimal solutions to subproblem instances,
    - and then combining these optimal subproblem solutions.

STEP 2: A RECURSIVE SOLUTION

- $m[i,j]$
  - ✓ the minimum number of scalar multiplications needed to compute $A_i \dots A_j$.
- **final solution:** $m[1,n]$
- **Assume that the optimal parenthesization splits the product $A_i A_{i+1} \dots A_j$ between A_k and A_{k+1}, where $i \le k < j$.**
- **Recursively,**
  - ✓ if $i = j$, $m[i,j] = 0$. (trivial)
  - ✓ if $i < j$, $m[i,j] = m[i,k] + m[k+1,j] + p_{i-1}p_k p_j$ for some k

where size of matrix $A_i = p_{i-1}\, p_i$.



A RECURSIVE SOLUTION

- **The minimum cost of parenthesizing the product** $A_i A_{i+1} \dots A_j$
  - ✓ $m[i,j] = \begin{cases} 0 & \text{if } i = j \\ \min\limits_{i \le k < j} (m[i,k] + m[k+1,j] + p_{i-1}p_k p_j) & \text{if } i < j \end{cases}$

- $s[i,j]$
  - ✓ a value k such that $m[i,j] = m[i,k] + m[k+1,j] + p_{i-1}p_k p_j$
  - ✓ It remembers where optimal parenthesization splits the product.

COMPUTING THE OPTIMAL COSTS

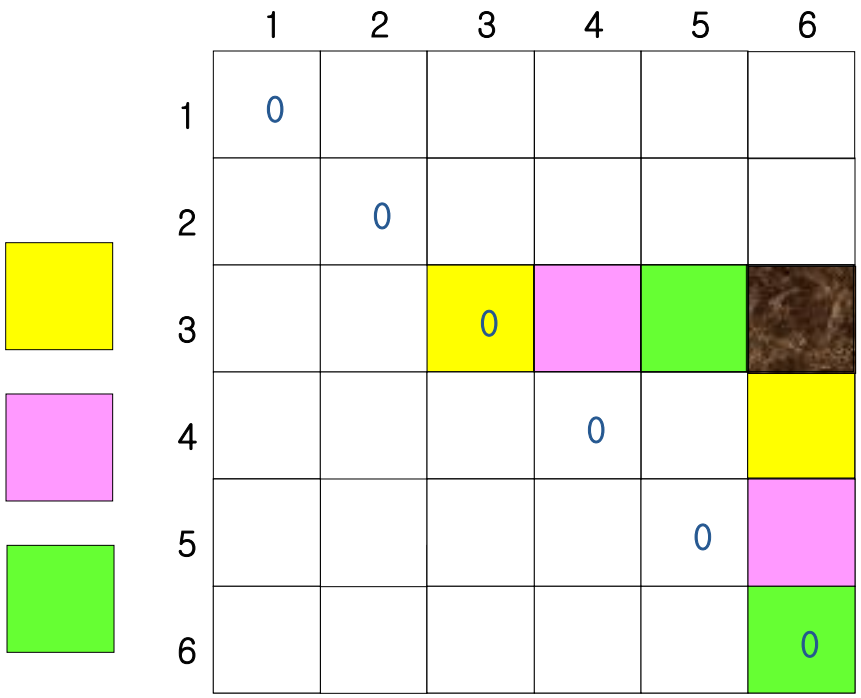
$$i = 3,\ \ j = 6,\ \ k = 3, 4, 5$$

$$\min_{i \le k < j} (m[i,k] + m[k+1,j] + p_{i-1}p_k p_j)$$

$$k = 3 : m[3,3] + m[4,6] + p_2 p_3 p_6$$

$$k = 4 : m[3,4] + m[5,6] + p_2 p_4 p_6$$

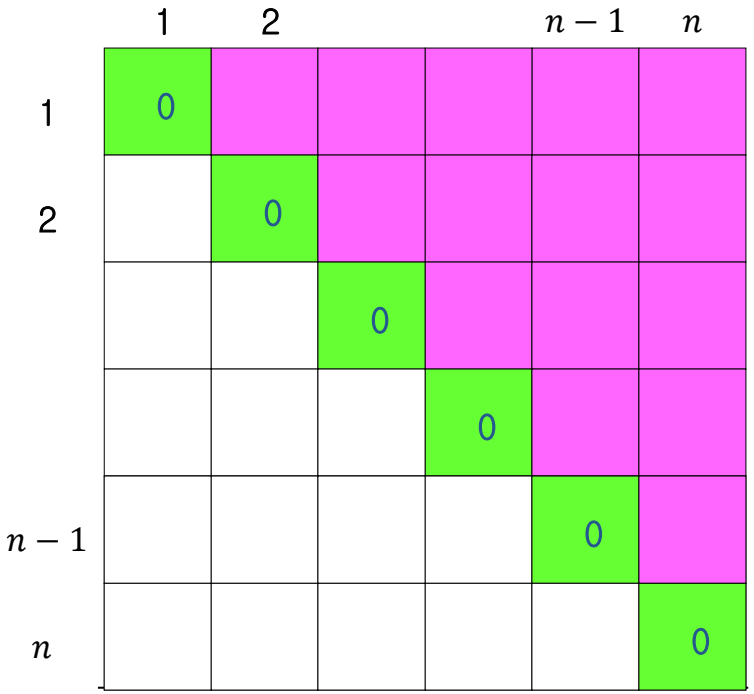
$$k = 5 : m[3,5] + m[6,6] + p_2 p_5 p_6$$



$m[1..n, 1..n]$

STEP 3: COMPUTING THE OPTIMAL COSTS

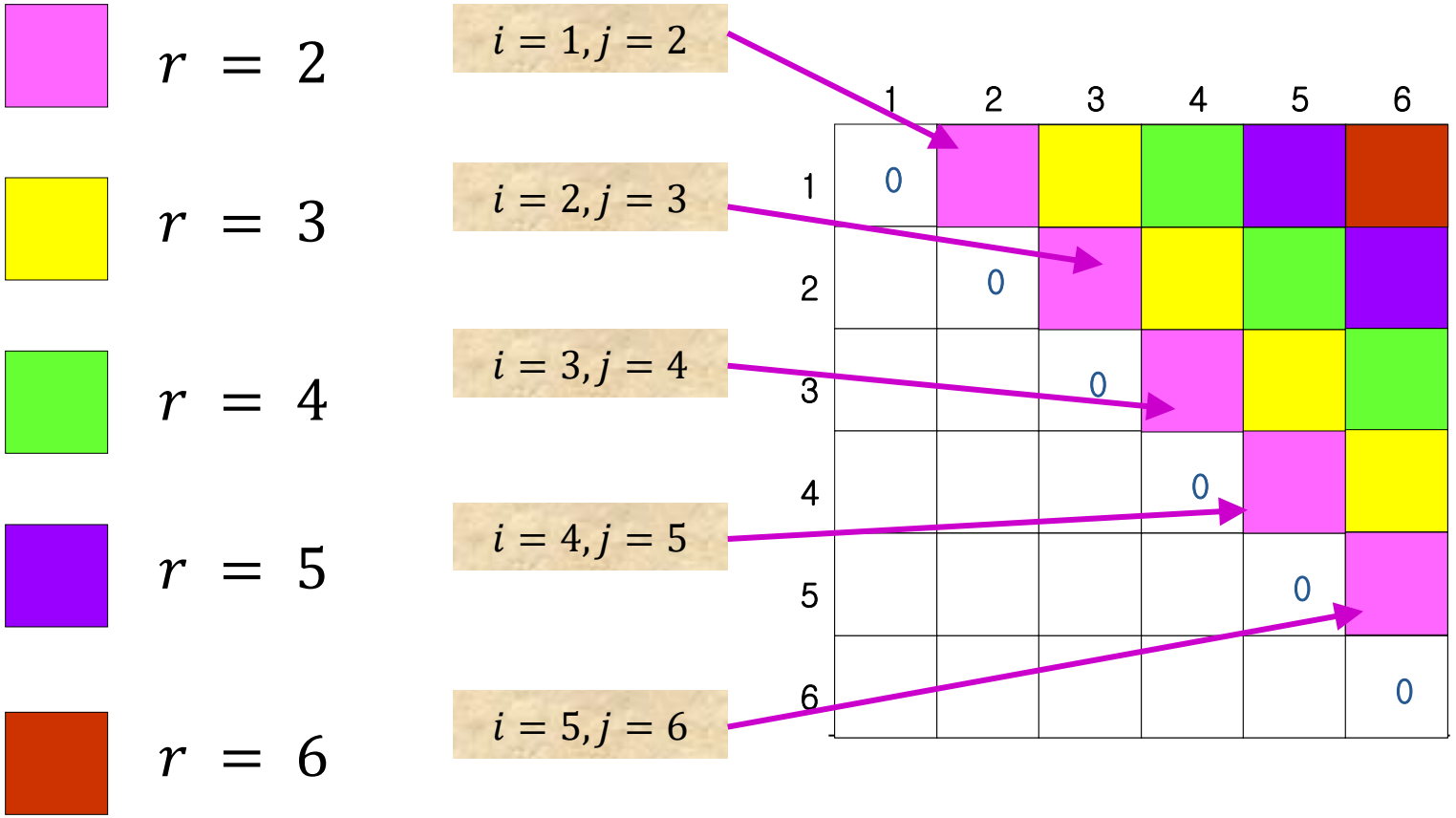
- **Resources**
  - Input (matrix dimensions)
    - $p = < p_0, p_1, \dots, p_n >$
  - Auxiliary table
    - $m[1..n, 1..n]$
    - $s[1..n, 1..n]$
- # of subproblems
  - ${}_nC_2 + n = \Theta(n^2)$
- Constructing the table has the time complexity $O(n^3)$.



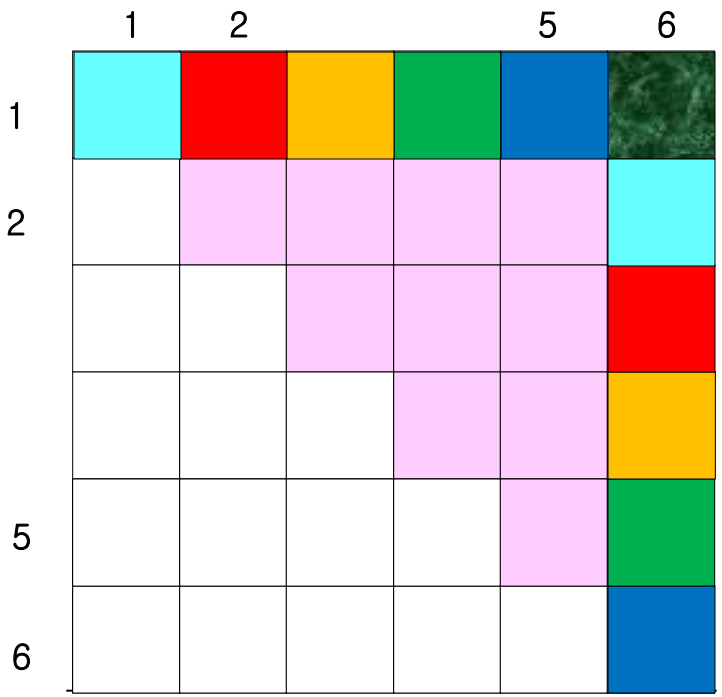
STEP 3: COMPUTING THE OPTIMAL COSTS

```
MATRIX-CHAIN-ORDER (p) {
  n ← length[p] - 1
  for i ← 1 to n
    m[i,i] ← 0
  for r ← 2 to n // r is the chain length.
    for i ← 1 to n-r+1
      j ← i + r - 1
      m[i,j] ← inifinity
      for k ← i to j-1
        q ← m[i,k] + m[k+1,j] + p[i-1]p[k]p[j]
        if q < m[i,j]
          m[i,j] ← q
          s[i,j] ← k
  return m and s
}
```

STEP 3: COMPUTING THE OPTIMAL COSTS



STEP 3: COMPUTING THE OPTIMAL COSTS



$$[1,6] = [1,5] \times [6,6]$$

$$[1,6] = [1,4] \times [5,6]$$

$$[1,6] = [1,3] \times [4,6]$$

$$[1,6] = [1,2] \times [3,6]$$

$$[1,6] = [1,1] \times [2,6]$$

STEP 4 : CONSTRUCTING AN OPTIMAL SOLUTION

Recall that $s[i,j]$ records the value of k s.t. the optimal parenthesization of $A_i A_{i+1} \dots A_j$ splits the product between A_k and A_{k+1}.

```
PRINT-OPTIMAL-PARENS(s,i,j){
  if i=j
    then print “A_i”
  else {
    print “(“
    PRINT-OPTIMAL-PARENS(s, i, s[i,j])
    PRINT-OPTIMAL-PARENS(s, s[i,j]+1, j)
    print “)“
  }
}
```

EXAMPLE: $A_1\ A_2\ A_3\ A_4\ A_5\ A_6$

matrix	dimesion
A_1	30×35
A_2	35×15
A_3	15×5
A_4	5×10
A_5	10×20
A_6	20×25

Step 1: $m_{i,i} = 0$: line 2,3

$m_{i,j}$

i \ j	1	2	3	4	5	6
1	0					
2		0				
3			0			
4				0		
5					0	
6						0

EXAMPLE: $A_1\ A_2\ A_3\ A_4\ A_5\ A_6$

matrix	dimesion
A_1	30×35
A_2	35×15
A_3	15×5
A_4	5×10
A_5	10×20
A_6	20×25

Step 2: $m_{i,i+1} = p_{i-1}p_ip_{i+1}$: $r = 2$

$m_{i,j}$

i \ j	1	2	3	4	5	6
1	0	15750				
2		0	2625			
3			0	750		
4				0	1000	
5					0	5000
6						0

EXAMPLE: $A_1\ A_2\ A_3\ A_4\ A_5\ A_6$

matrix	dimesion
A_1	30×35
A_2	35×15
A_3	15×5
A_4	5×10
A_5	10×20
A_6	20×25

Step 3: $m_{i,i+2}$: $r = 3$

$m_{i,j}$

i \ j	1	2	3	4	5	6
1	0	15750				
2		0	2625			
3			0	750		
4				0	1000	
5					0	5000
6						0

Step 4: $m_{i,i+3}$: $r = 4$

Step 5: $m_{i,i+4}$: $r = 5$

Step 6: $m_{i,i+5}$: $r = 6$

EXAMPLE: $A_1\ A_2\ A_3\ A_4\ A_5\ A_6$

Step 2:

$m_{i,j}$

i \ j	1	2	3	4	5	6
1	0	15750				
2		0	2625			
3			0	750		
4				0	1000	
5					0	5000
6						0

$s_{i,j}$

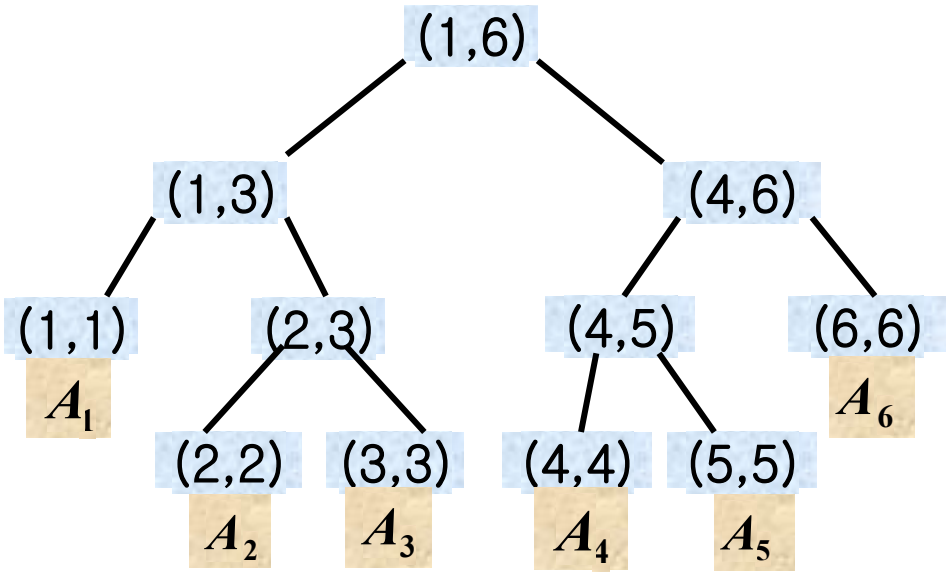
i \ j	1	2	3	4	5	6
1	0	1				
2		0	2			
3			0	3		
4				0	4	
5					0	5
6						0

EXAMPLE: $A_1\ A_2\ A_3\ A_4\ A_5\ A_6$

$s_{i,j}$ (i \ j)	1	2	3	4	5	6
1	0	1	1	3	3	3
2		0	2	3	3	3
3			0	3	3	3
4				0	4	5
5					0	5
6						0

```
PRINT-OPTIMAL-PARENS(s,i,j){
  if i=j
    then print "A_i"
  else {
    print "("
    PRINT-OPTIMAL-PARENS(s, i, s[i,j])
    PRINT-OPTIMAL-PARENS(s, s[i,j]+1, j)
    print ")"
  }
}
```

parenthesization tree



$$((A_1\ (A_2\ A_3))\ ((A_4\ A_5)\ A_6))$$

EXAMPLE: $A_1\ A_2\ A_3\ A_4\ A_5\ A_6$

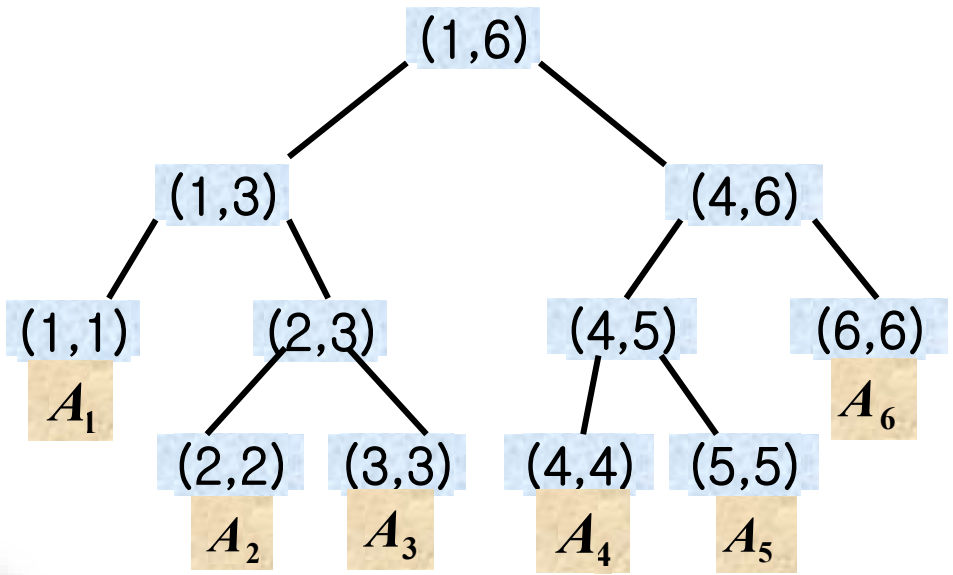
❖ **What is the time complexity to get the parenthesization tree?**

$s_{i,j}$

i \ j	1	2	3	4	5	6
1	0	1	1	3	3	3
2		0	2	3	3	3
3			0	3	3	3
4				0	4	5
5						
6						

```
PRINT-OPTIMAL-PARENS(s,i,j){
  if i=j
    then print "A_i"
  else {
    print "("
    PRINT-OPTIMAL-PARENS(s, i, s[i,j])
    PRINT-OPTIMAL-PARENS(s, s[i,j]+1, j)
    print ")"
  }
}
```

parenthesization tree



$$((A_1\ (A_2\ A_3))\ ((A_4\ A_5)\ A_6))$$

CONCLUSION

- **For an n-matrix chain with given dimensions $(p_0, p_1, \dots, p_n)$**
  - ✓ Our goal is to find the optimal parenthesization.
  - ✓ However, the total number of possible parenthesizations is $\Omega(2^n)$, which grows exponentially.
  - ✓ Therefore, it is not feasible to try all possible cases and select the best one.

  - ✓ We observe that the problem has **overlapping subproblems**!
  - ✓ To avoid redundant computations, we apply **dynamic programming (DP)**.
  - ✓ We construct tables to store:
    - the minimum number of multiplications, and
    - the optimal split positions.
  - ✓ Filling these table takes $O(n^3)$ time.
  - ✓ Once these tables are computed, you can quickly $(\Theta(n))$ find the best way to parenthesize the n-matrix chain.
- **Final Result**
  - ✓ The overall time complexity is $O(n^3)$, polynomial time, which is **much more efficient** than the brute-force approach $\Omega(2^n)$, exponential.

Pop Quiz

❖ Pop Quiz (2 point)

- ✓ Draw the parenthesization tree for $i = 2, j = 6$ according to the table $s[i,j]$
- ✓ Upload your answer (drawing) to the HDLMS [Quizzes] menu.
  - Quiz title: "(Quiz 01) DP"

$s[i,j]$

i \ j	1	2	3	4	5	6
1	0	1	1	3	3	3
2		0	2	3	3	**3**
3			0	3	3	3
4				0	4	5
5					0	5
6						0

Thank you!